SVARA: KARAOKE AND LEARN

An Integrated Vocal Analysis, Karaoke and Guided Singing Practice Application

A PROJECT REPORT

Submitted by

MAHATHI B - 241701029

SNEHA S – 241701055

In fulfilment for the course

GE23627 – DESIGN THINKING AND INNOVATION

for the degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND DESIGN



RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602 105

BONAFIDE CERTIFICATE

Certified that this Report titled "SVARA: KARAOKE AND LEARN – AN INTEGRATED VOCAL ANALYSIS, KARAOKE AND GUIDED SINGING PRACTICE APPLICATION" is the bonafide work of MAHATHI B (241701029) and SNEHA S (241701029), who carried out the work under my supervision.

Certified further that the project contributes to SDG 4, SDG 9 and SDG 10, with a primary focus on SDG 4 – Quality Education, through the development of an integrated vocal analysis, karaoke and music-learning system that promotes accessible music education, personalised vocal development and technology-enabled learning.

SIGNATURE OF HOD

Dr. S. Vinodh Kumar

Professor, The Head of the Department

Computer Science and Design

Rajalakshmi Engineering College

Chennai – 602105

SIGNATURE OF SUPERVISOR

Dr. P. Revathy

Professor

Computer Science and Design

Rajalakshmi Engineering College Chennai – 602105

ABSTRACT

Learning to sing involves listening, pitch matching, breath control, repeated practice and timely feedback. These activities are often supported by separate tools for karaoke playback, vocal exercises, lyric display, pitch analysis and recording. Moving between disconnected tools can interrupt practice and make it difficult for learners to relate an initial vocal assessment to later exercises. SVARA: Karaoke and Learn is designed as an integrated mobile application that brings these activities into a common practice environment. The application is developed with a Flutter-based mobile frontend and a Python backend using FastAPI for audio-processing operations. Supabase is used for authentication and data persistence. The audio workflow uses established tools and pretrained models, including Demucs for music source separation, a UVR/MDX-Net-family model for further vocal processing, librosa pYIN for reference-pitch estimation, and faster-whisper for timestamped lyric transcription. A custom YIN-based method is used in the song-learning workflow to estimate the learner's pitch from microphone input. SVARA is organised around vocal analysis, karaoke, song-based learning and guided exercises. The karaoke workflow prepares separated audio stems and provides playback controls. The learning workflow presents lyrics and reference-pitch information, records the learner's voice and provides pitch-oriented feedback. Vocal-profile information and practice records are intended to connect assessment with subsequent practice. The system is designed to support Tamil, Malayalam, Telugu, Hindi and English transcription workflows; however, sung-lyric recognition and timing may vary with language, pronunciation, accompaniment and recording conditions. The project's principal contribution is its integration of vocal assessment, song preparation and guided singing practice in a single mobile-oriented workflow. SVARA does not claim to introduce a newly trained audio model. User feedback and technical evaluation should be reported with the actual protocol and measured observations. No numerical pitch-accuracy, lyric-accuracy or learning-effect result is asserted in this draft unless supported by completed measurements.

ACKNOWLEDGEMENT

Initially, we thank the Almighty for being with us through every walk of our lives and showering His blessings throughout the endeavour to complete this report. Our sincere thanks to our Chairman Mr. S. Meganathan, B.E., F.I.E., our Vice Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., and our respected Chairperson Dr. (Mrs.) Thangam Meganathan, Ph.D., for providing us with the requisite infrastructure and facilities at their esteemed institution.

Our sincere thanks to Dr. S. N. Murugesan, M.E., Ph.D., our beloved Principal, for his kind support and the facilities provided to complete our project work successfully. We express our sincere thanks to Dr. S. Vinodh Kumar, Ph.D., Professor and Head of the Department of Computer Science and Design, for his valuable guidance and encouragement throughout the project. We convey our heartfelt gratitude to our internal project guide, Dr. P. Revathy, Professor, Department of Computer Science and Design, for her continuous guidance, constructive suggestions and valuable support during the development of our project. We also thank the faculty members, our families and friends for their encouragement and support.

MAHATHI B (241701029)

SNEHA S (241701055)

TABLE OF CONTENTS

S. NO.	TITLE	PAGE NO.
1	INTRODUCTION	
2	LITERATURE SURVEY	
3	SYSTEM DESIGN	
4	HARDWARE AND DEVELOPMENT ENVIRONMENT	
5	SOFTWARE IMPLEMENTATION	
6	METHODOLOGY AND ALGORITHMS	
7	TESTING AND RESULTS	
8	CONCLUSION AND FUTURE SCOPE	
	REFERENCES	

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Overall system architecture of SVARA	
3.2	Overall application workflow	
3.3	Use case diagram of SVARA	
3.4	Activity diagram for vocal-range assessment	
3.5	Module diagram of svara	
5.1	SVARA application homescreen	
5.2	Karaoke mixer of SVARA	
5.3	*Learning screen with lyrics and pitch visualisation*	
5.4	Profile authentication	

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing singing-practice applications and SVARA	
3.1	Functional requirements	
3.2	Non-functional requirements	
4.1	Software and development tools	
6.1	Pitch-analysis parameters used in the implementation	
7.1	Functional test cases	
7.2	Summary of user feedback	

LIST OF ABBREVIATIONS

S. NO.	ABBREVIATION	EXPANSION
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	ASR	Automatic Speech Recognition
4	CPU	Central Processing Unit
5	F0	Fundamental Frequency
6	FFT	Fast Fourier Transform
7	GUI	Graphical User Interface
8	HTTP	Hypertext Transfer Protocol
9	MIDI	Musical Instrument Digital Interface
10	ML	Machine Learning
11	RMS	Root Mean Square
12	SDK	Software Development Kit
13	STFT	Short-Time Fourier Transform
14	UI	User Interface
15	UX	User Experience
16	UVR	Ultimate Vocal Remover
17	YIN	Fundamental-frequency estimation algorithm
18	pYIN	Probabilistic YIN
19	REST	Representational State Transfer
20	DB	Database

1. INTRODUCTION

1.1 General

Singing is a skill that combines auditory perception, pitch control, breath management, vocal coordination and regular practice. A learner must be able to hear a musical reference, understand the notes being performed, reproduce those notes and recognise differences between the reference and their own voice. Beginners may find it difficult to identify whether a note is too high or too low, determine a comfortable vocal range or practise a song in manageable sections.

Digital tools can support these activities through karaoke tracks, pitch displays, recording facilities and guided exercises. However, these features are commonly spread across different applications. SVARA: Karaoke and Learn aims to bring vocal assessment, karaoke preparation and guided singing practice into one mobile application.

1.2 Background

Digital audio processing makes it possible to analyse recorded sound, estimate fundamental frequency, separate musical sources and generate timestamped transcriptions. Mobile devices also provide microphones, speakers and sufficient processing capability to support interactive practice. Cloud and backend services can coordinate computationally intensive tasks, while a mobile frontend provides the learner with a consistent interface.

SVARA uses these technologies as components of a connected workflow. The application is not intended to replace a qualified vocal instructor. Instead, it provides visual and audio-based practice assistance that can help users repeat tasks and observe selected aspects of their singing.

1.3 Problem Statement

Learners who practise independently may not have a structured way to assess their vocal range, isolate a song's accompaniment, follow lyrics in time with playback, compare their pitch with a reference or revisit previous recordings. Using multiple

applications can interrupt the practice process and require repeated importing, exporting and manual configuration.

There is a need for an integrated and accessible application that connects vocal assessment, karaoke audio preparation, lyric-supported song practice, pitch-oriented feedback and guided exercises within a single user workflow.

1.4 Motivation

The motivation for SVARA is to make structured singing practice more accessible to learners who may not have continuous access to formal music training. A learner should be able to explore their approximate vocal range, practise with a song, listen to the instrumental accompaniment, follow lyrics and receive immediate pitch-related visual feedback. The project also provides an opportunity to combine mobile application development, backend engineering and digital signal processing in a practical system.

1.5 Objectives

- **To develop a mobile application that combines vocal analysis, karaoke and guided singing practice.**
- **To estimate an approximate vocal range through repeated pitch-detection tasks.**
- To prepare karaoke audio by separating a song into usable stems and providing playback controls.
- To generate timestamped lyrics for supported language workflows and present them alongside song playback.
- To estimate reference pitch and compare it with the learner's microphone input during practice.
- To provide understandable pitch-oriented feedback and allow users to record and revisit practice takes.
- To provide guided vocal exercises and maintain relevant profile or practice information.

1.6 Scope and Limitations

The scope includes a Flutter mobile frontend, a Python/FastAPI audio-processing backend and Supabase-based authentication and persistence. The principal workflows include vocal analysis, karaoke stem preparation, song-based learning, pitch feedback, recording and guided exercises. The application uses established signal-processing algorithms and pretrained models.

The system's output depends on microphone quality, background noise, audio mixing, vocal technique, accompaniment and model behaviour. Pitch estimation can produce octave errors or unstable values, particularly during breathy, vibrato-rich or overlapping audio. Automatic transcription of sung lyrics may contain recognition and timing errors. Any automatically inferred voice category is an approximate profile hint and not a medical or formal pedagogical classification.

1.7 Existing System

Existing singing-practice workflows may involve a combination of karaoke applications, audio stem-separation tools, lyric websites, pitch-monitoring applications, voice recorders and exercise resources. Each tool can provide a useful function, but the learner may need to move between them to complete a single practice session.

1.8 Limitations of Existing Approaches

- Fragmented workflows require learners to switch between multiple applications.
- Some karaoke tools provide accompaniment but do not connect playback to a learner-specific pitch-analysis workflow.
- Standalone pitch visualisers may not provide song lyrics, reference tracks or structured exercises.
- Manual lyric entry and manual audio preparation can increase setup time.
- Progress and recordings may remain distributed across separate tools.

1.9 Proposed SVARA System

SVARA proposes a unified workflow in which users can begin with a vocal assessment, prepare a song for karaoke, enter a learning session and practise with visual pitch guidance. The backend coordinates audio-processing tasks such as source separation, transcription and reference-pitch extraction. The mobile application handles playback, recording, user interaction and presentation of analysis results. Supabase supports account-related and practice data persistence.

1.10 Advantages and Applications

- Combines vocal assessment, karaoke and learning-oriented practice in one application.
- Provides configurable audio playback and stem controls for song practice.
- Connects lyric display and reference-pitch information with microphone-based practice.
- Supports repeated practice and recording review.
- Can be used by beginners, hobby singers and learners who want visual assistance between lessons.

2. LITERATURE SURVEY

2.1 Introduction

SVARA draws on research and engineering developments in fundamental-frequency estimation, computer-assisted singing, music source separation, automatic speech recognition and mobile learning systems. These areas provide the technical foundation for the application's processing pipeline. This chapter summarises the relevant concepts and explains how they relate to the proposed system.

2.2 Fundamental-Frequency Estimation

Fundamental frequency (F0) is the rate of repetition of a periodic signal and is commonly associated with the perceived pitch of a voiced sound. In singing analysis, F0 estimation is used to represent a vocal performance as a sequence of pitch values over time. The YIN algorithm estimates F0 using a difference function and a cumulative mean normalised difference function. Probabilistic YIN (pYIN) extends the approach by considering multiple pitch candidates and temporal decoding.

SVARA uses a custom YIN-based method for user-pitch estimation in the song-learning workflow and librosa pYIN for song-derived reference pitch. These methods provide useful pitch tracks but are not error-free. Background accompaniment, harmonics, vibrato, breathiness and overlapping voices can affect estimates.

2.3 Computer-Assisted Singing Practice

Computer-assisted singing systems use visual or auditory feedback to help learners observe aspects of their performance. A pitch trace can make the difference

between a target note and a performed note more visible. Such feedback may support awareness and repetition, but the design of a feedback display does not by itself establish improved singing ability. The effectiveness of any particular scoring threshold or practice method requires evaluation with suitable participants and reference measurements.

2.4 Music Source Separation

Music source separation attempts to decompose a mixed audio signal into constituent sources such as vocals, drums, bass and other accompaniment. Demucs is a neural-network-based music source-separation system that can produce separated stems from a mixed recording. A further UVR/MDX-Net-family vocal-processing stage is used for additional processing of the vocal stem. Separating a complete vocal stem from accompaniment is not the same as reliably identifying every lead, harmony and ad-lib singer. Any finer vocal-part division that uses thresholds or activity heuristics must be described as heuristic processing rather than a validated semantic singer-separation model.

2.5 Automatic Speech Recognition and Lyrics

Automatic speech recognition converts audio into text and may provide timestamps for recognised words. faster-whisper is used in SVARA to generate timestamped transcriptions from isolated vocal audio. Singing differs from ordinary speech because of sustained vowels, melodic pitch movement, instrumental overlap, pronunciation variation and expressive timing. Consequently, transcription quality and timestamp alignment sung material. Recognition should not be described as perfect for any supported language.

2.6 Mobile Audio Applications and Data Persistence

A mobile application can combine audio playback, microphone capture, visual feedback and account-based data. Flutter provides a cross-platform application framework, while FastAPI can expose backend processing operations through HTTP endpoints. Supabase provides authentication, relational data storage and object storage facilities. These technologies support the application architecture, but security, data ownership, storage permissions and error handling must be tested as part of deployment.

2.7 Comparative Analysis

System / Approach	Primary Function	Typical Limitation / SVARA Context
Karaoke applications	Playback with lyrics or instrumental accompaniment	May not combine vocal-range assessment and reference-versus-user pitch analysis.
Audio stem-separation tools	Separate vocals and accompaniment	Usually focus on audio processing rather than a complete learner workflow.
Pitch visualisers	Display estimated pitch or tuning information	May not include song preparation, lyrics and guided practice.
Voice recorder applications	Capture and replay user recordings	Recording alone does not provide reference pitch or structured analysis.
SVARA	Integrated vocal analysis, karaoke and guided practice	Combines established components; its accuracy and educational impact require systematic evaluation.

2.8 Research Gap

The practical opportunity addressed by SVARA is not the invention of a new pitch-estimation or source-separation algorithm. Rather, it is the integration of assessment, song preparation, lyric-supported practice, visual pitch feedback and recording within one mobile-facing workflow. The project connects these separate processing stages so that a learner can move from an approximate vocal profile to song practice and guided exercises without repeatedly changing applications.

2.9 Summary

The literature provides established methods for pitch estimation, source separation and transcription. SVARA applies these methods within an integrated application and must distinguish the capabilities of the underlying models from the measured performance of the complete product.

3. SYSTEM DESIGN

3.1 General

System design defines the major components, interactions and data flow required to implement SVARA. The system is organised into a Flutter frontend, a FastAPI backend and a Supabase data layer. Audio-processing components are invoked by the backend, while the frontend provides the learner-facing workflow.

3.2 Requirement Analysis

The application must allow a user to navigate the major modules, provide audio input, initiate processing, play the resulting audio, record microphone input and view relevant analysis. The backend must coordinate long-running audio jobs and return processing results in a format that the application can use. The data layer must support account and practice information.

3.3 Functional Requirements

ID	Requirement
FR-01	The user can access the vocal-analysis workflow and perform guided pitch tasks.
FR-02	The user can import a song for karaoke processing.
FR-03	The backend can separate the input song into audio stems using the configured processing pipeline.
FR-04	The user can control playback and available stem levels.
FR-05	The learning workflow can request lyrics and reference-pitch information for a song.
FR-06	The user can record microphone audio during practice.
FR-07	The application can display reference

	and user pitch information for comparison.
FR-08	The user can access guided exercises and relevant profile information.
FR-09	Account and practice information can be stored using the configured persistence layer.

3.4 Non-Functional Requirements

Category	Requirement
Usability	Provide clear navigation and understandable practice feedback.
Responsiveness	Show processing state and handle long-running audio jobs without confusing the user.
Reliability	Handle invalid audio, unavailable services and microphone errors gracefully.
Maintainability	Separate frontend, backend and audio-processing responsibilities.
Security	Protect user accounts, recordings and processing jobs through appropriate access control.
Portability	Support the intended mobile deployment environment.

3.5 Overall System Architecture

The Flutter frontend is responsible for application navigation, audio selection, playback controls, microphone capture, pitch visualisation and presentation of results. The FastAPI backend receives processing requests and orchestrates the audio pipeline. Demucs performs broad source separation; a UVR/MDX-Net-family model performs additional vocal processing; librosa pYIN estimates reference pitch; faster-whisper generates timestamped transcription; and FFmpeg

supports audio conversion and file operations. Supabase provides authentication and persistence for supported user and practice data.

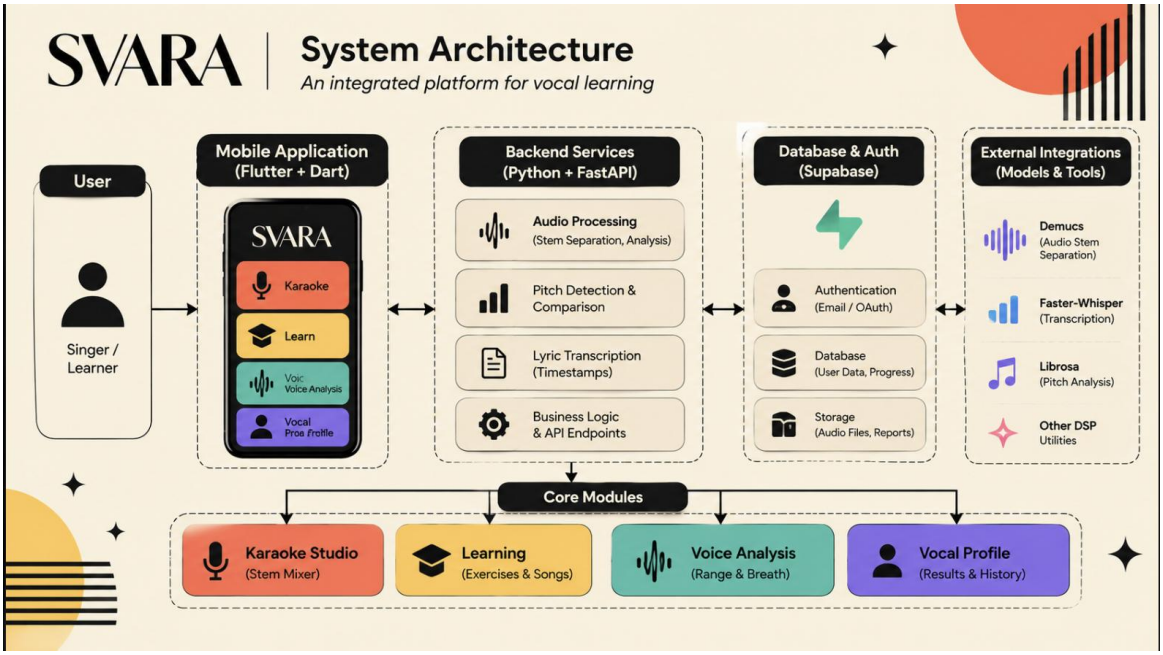


Figure 3.1 Overall system architecture of SVARA

3.6 Overall System Workflow

A typical song-learning workflow begins when the user selects an audio file. The frontend submits the file or a processing request to the backend. The backend checks whether a reusable cached result is available using a SHA-256-based content key. If processing is required, the audio is separated into stems and the vocal audio is passed to the relevant pitch and transcription stages. The frontend then presents playback, lyrics and reference-pitch information. During practice, the microphone captures the learner's voice and the pitch-analysis component estimates the user's pitch for visual comparison and feedback.

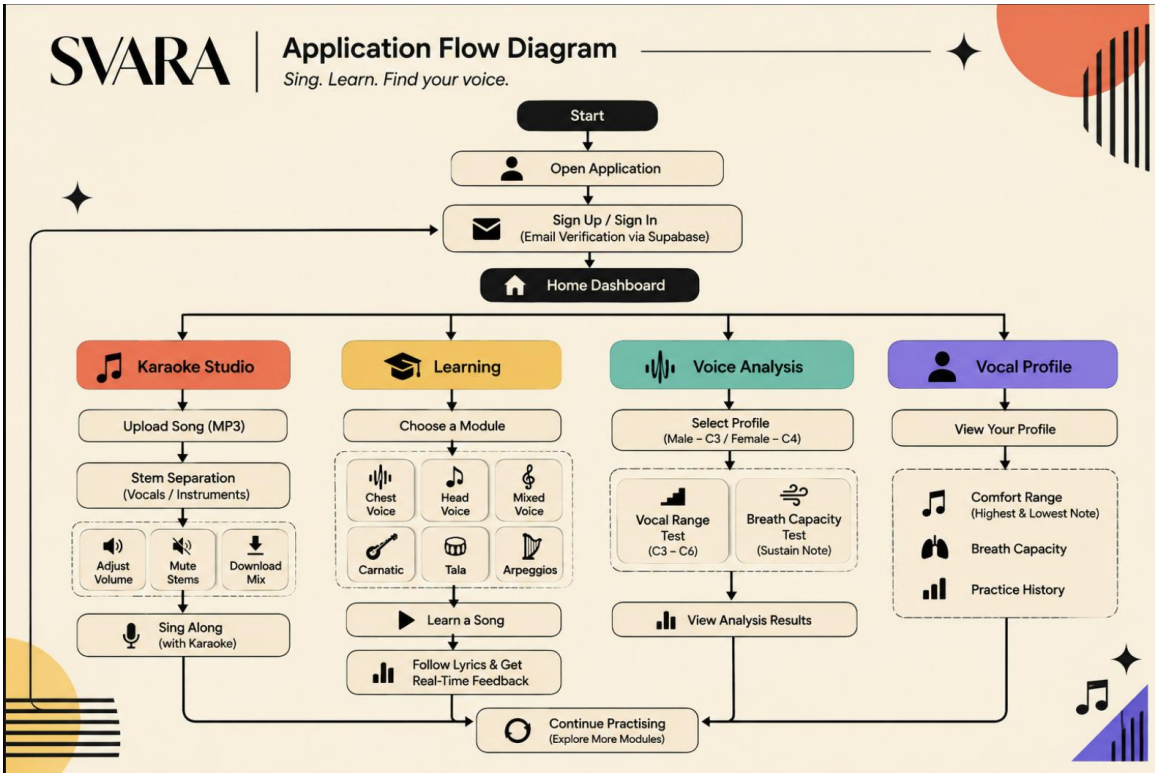


Figure 3.2 Overall system flow diagram

3.7 Use Case Design

The primary actor is the application user. The user can create or access an account, view a vocal profile, perform vocal analysis, import a song, generate karaoke stems, adjust playback, open a learning session, follow lyrics, record singing, view pitch feedback and access guided exercises. Backend services perform audio processing and data operations in response to application requests.

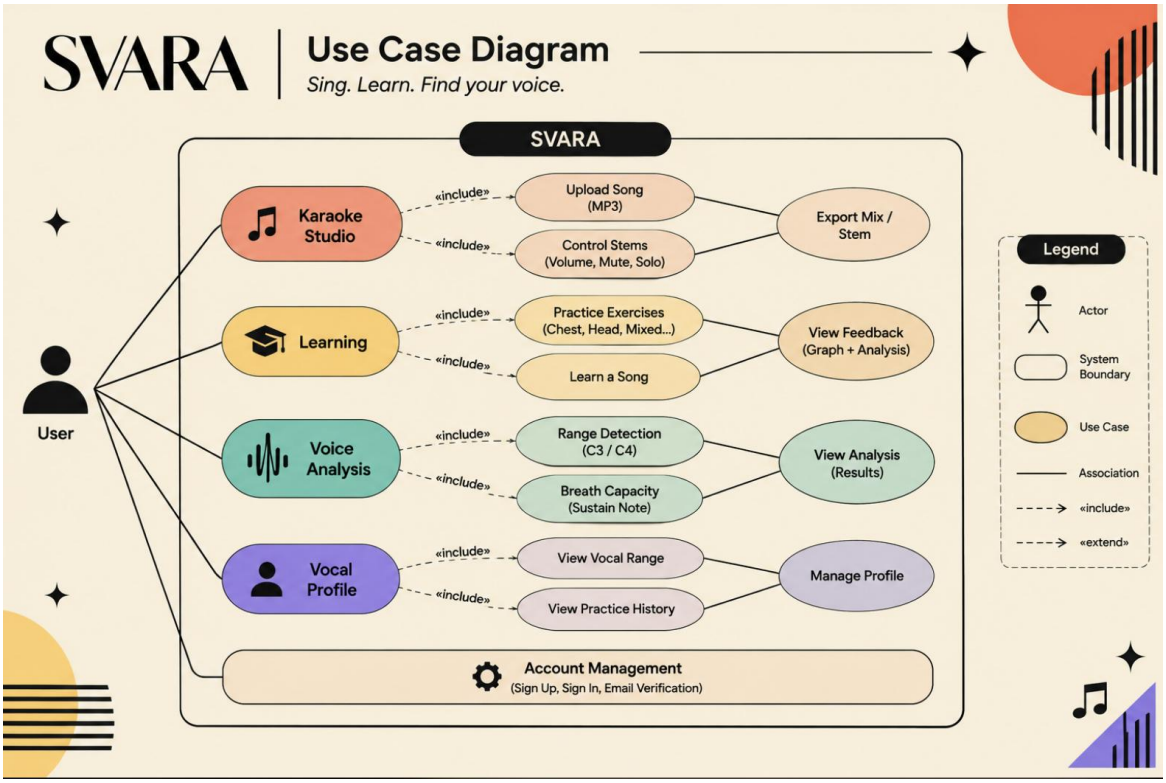


Figure 3.3 Use case diagram of SVARA

3.8 Activity and Sequence Diagrams

The vocal-analysis activity begins with a guided prompt and a target note. The application captures microphone input, estimates pitch and checks whether the detected note satisfies the configured acceptance condition. The process continues across the selected range until the scan is complete or the configured limits are reached. The song-learning sequence begins with song import, backend processing, reference extraction, lyric generation, playback, microphone recording and pitch comparison.

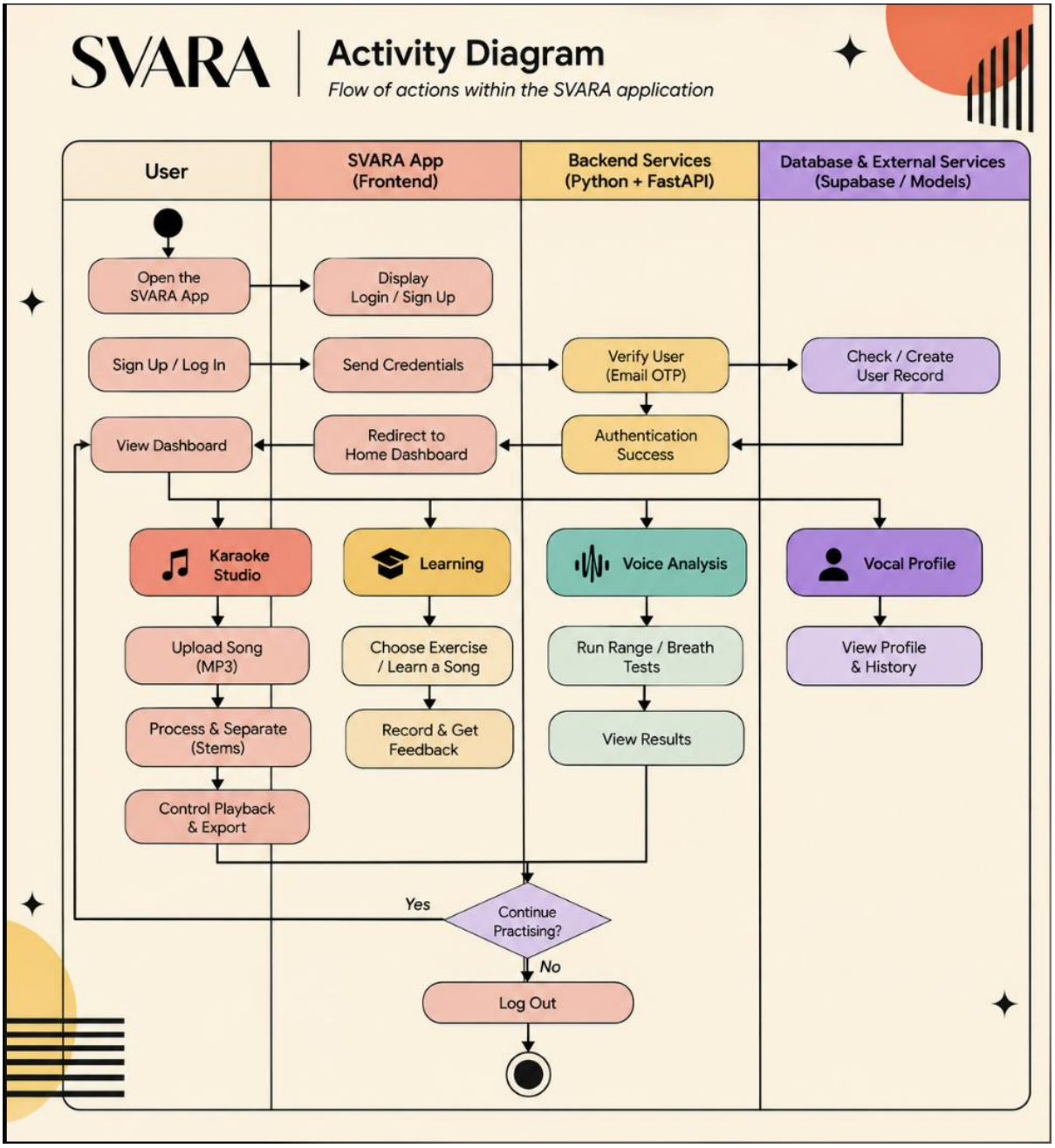


Figure 3.4 Activity diagram for the CHROMAPOD scanning workflow

3.9 Audio Processing and Data Flow

The backend processing pipeline includes input validation, content hashing, cache lookup, source separation, additional vocal processing, reference-pitch extraction and lyric transcription. The output files and metadata are returned to the frontend for playback and visualisation. User recordings and profile information follow separate data paths and should be associated with the authenticated account.

3.10 Module Design

- Vocal Analysis Module: guides the user through pitch tasks and estimates an approximate low-to-high vocal range.
- Karaoke Module: imports songs, requests stem separation and provides playback and mixer controls.
- Learning Module: combines lyrics, reference pitch, user recording and pitch-oriented feedback.
- Exercise Module: presents structured singing and breathing-related practice activities.
- Profile and Persistence Module: manages account-linked profile information and available practice records.

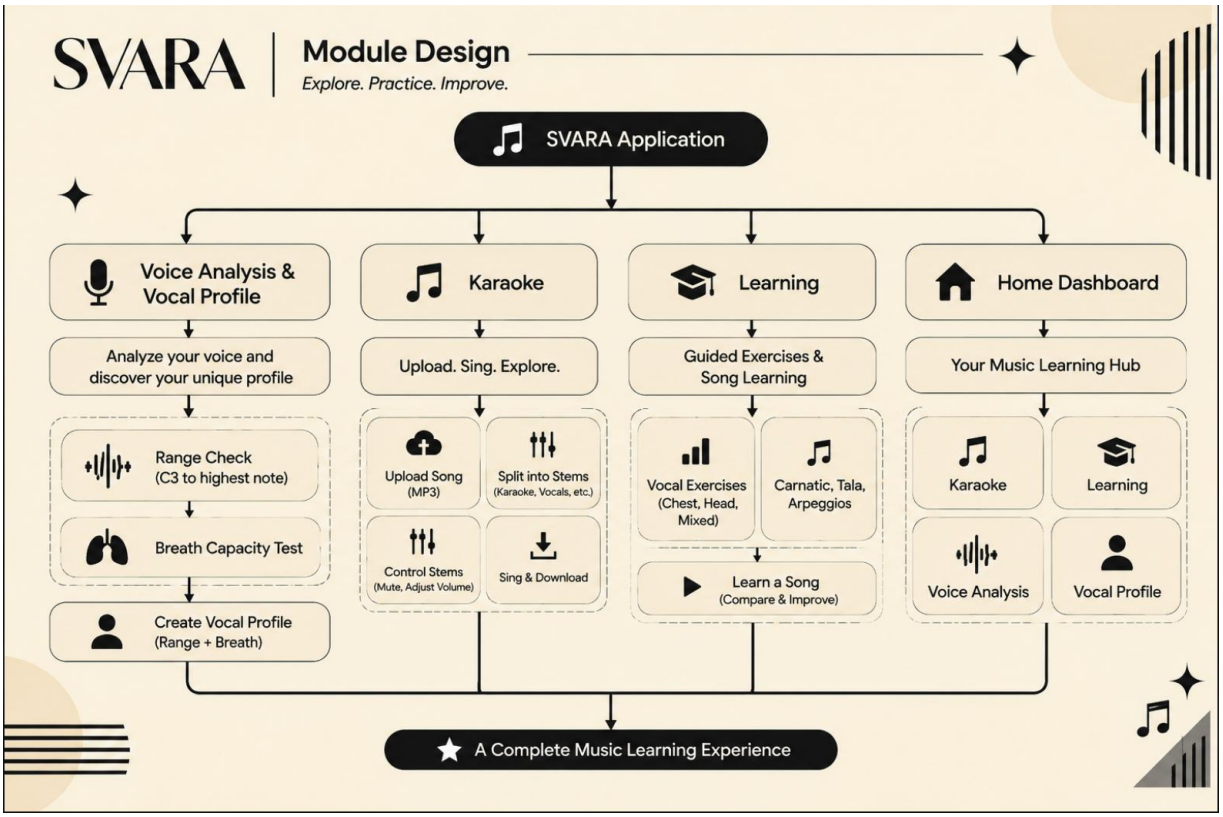


Figure 3.5 Module design for SVARA

4. HARDWARE AND DEVELOPMENT ENVIRONMENT

4.1 General

SVARA is primarily a software-based mobile application and does not require a custom external hardware prototype. The intended user-facing device is an Android smartphone or another supported mobile device with a microphone, speaker or headphones and sufficient storage for audio files. A development computer is used to build the mobile application and run backend audio-processing services.

4.2 Mobile Device Requirements

The mobile device provides the microphone input required for vocal analysis and practice recording. Its speaker or connected headphones provide song playback and reference audio. Microphone permissions must be requested and handled appropriately. Recording quality may vary with device hardware, microphone placement, room acoustics and background noise.

4.3 Development Computer

The development computer is used for Flutter application development, Python backend execution, model inference, debugging and testing. Audio-processing operations may require substantial CPU, memory and storage resources depending on the selected model and input duration.

4.4 Software and Development Tools

Tool / Technology	Role in SVARA
Flutter / Dart	Mobile frontend, navigation, playback interface and user interaction.
Python	Backend processing and audio-analysis logic.
FastAPI	HTTP API layer for audio-processing requests.
Supabase	Authentication, database and configured

	storage services.
Demucs	Music source separation into broad stems.
UVR / MDX-Net-family model	Additional processing of the vocal stem.
librosa pYIN	Reference fundamental-frequency estimation.
faster-whisper	Timestamped transcription from vocal audio.
YIN-based pitch analysis	User-pitch estimation in the song-learning workflow.
FFmpeg	Audio conversion and file-processing support.
Git / GitHub	Version control and source-code collaboration.

4.5 System Requirements

The application requires access to audio files, microphone input, network connectivity for backend operations and sufficient local storage for temporary and saved recordings. The backend requires the relevant Python packages, model files or model access, FFmpeg and writable temporary storage. Exact versions and model identifiers should be recorded in the final implementation appendix for reproducibility.

4.6 Hardware and Software Integration

The smartphone frontend communicates with the backend through API requests. Audio files and processing parameters are transferred to the backend, and the backend returns job status, output locations and analysis data. The frontend then loads the required audio assets and presents the user interface. Supabase is used for the account and data layer according to the implemented schema and access policies.

5. SOFTWARE IMPLEMENTATION

5.1 Software Architecture and Technology Stack

SVARA follows a layered software architecture. The Flutter frontend handles presentation and user interaction. The FastAPI backend coordinates computational audio tasks. Audio-processing libraries and pretrained models perform separation, pitch extraction and transcription. Supabase supports authentication and persistent application data. This separation allows the mobile interface to remain focused on the learner experience while heavier processing is delegated to backend services.



5.1 svara home screen

5.2 Flutter Frontend

The Flutter application provides navigation between the main modules and handles audio import, playback, microphone recording, visualisation and user interaction. The frontend communicates with backend endpoints to initiate audio processing and retrieve results. The user interface is designed to make technical audio information understandable through clear controls, lyrics and pitch displays.

5.3 Python and FastAPI Backend

The Python backend exposes API routes for audio-processing operations. FastAPI is used to receive requests, validate parameters and coordinate processing services. Long-running operations such as source separation and transcription are handled by backend processing tasks. The backend returns processing status and result metadata for use by the frontend. Production deployment requires robust job ownership, authentication checks, file-path validation, resource limits and cleanup

5.4 Karaoke Module

The Karaoke module allows the user to import a song and prepare it for practice. The backend uses Demucs to generate broad source stems. The vocal stem may be processed further using a UVR/MDX-Net-family model. The application exposes the available audio components through a compact mixer, allowing the learner to adjust the relative levels and mute or unmute stems. Playback controls and supported transpose or speed options allow the user to configure the practice track.

Any lead, harmony or ad-lib separation beyond the broad source stems must be described according to the actual implementation. If a threshold-based activity split is used, it is a heuristic partition and should not be represented as a validated model that understands singer identity or musical role.



5.2 karaoke mixer of svara

5.5 Vocal Analysis Module

The Vocal Analysis module guides the user through a sequence of target-note tasks. The application captures microphone input and estimates the fundamental frequency of the voiced signal. A detected pitch is compared with the target note, and repeated successful detections are used to determine whether the note can be included in the estimated range. The lowest and highest accepted notes form an approximate vocal-range result.

The resulting profile should be interpreted as a practical estimate rather than a formal voice classification. Vocal comfort, tessitura, timbre, passaggio and expert assessment are not fully represented by range endpoints alone.

5.6 Learning Module

The Learning module is designed for song-based practice. After the user imports a song, the backend prepares the required vocal and accompaniment audio, extracts reference-pitch information and generates timestamped lyrics. The learner can follow the lyrics while listening to the reference or accompaniment. The user can record their own singing through the device microphone, and the application estimates the user's pitch for comparison with the reference.

The pitch display is intended to make pitch differences visible over time. Feedback may indicate whether a performed note is above, below or near the reference. These messages are practice cues and should not be interpreted as a validated measure of overall singing quality. The application should also make clear when pitch confidence is low or no reliable voiced frame is available.



5.3 Learning screen with lyrics and pitch visualisation

5.7 Lyrics and Multilingual Transcription

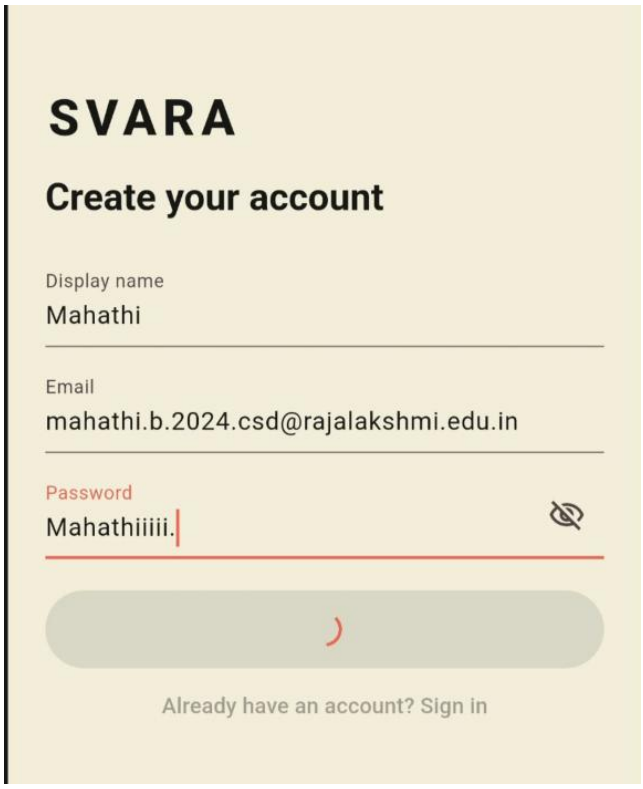
SVARA uses faster-whisper large-v3 on the isolated vocal audio to generate transcription with word-level timestamps. The intended language workflows include Tamil, Malayalam, Telugu, Hindi and English. The use of language hints can support transcription, but singing presents challenges that differ from conversational speech. The application should therefore present generated lyrics as machine-produced output that may require correction.

5.8 Guided Exercises

The exercise area provides structured practice activities intended to complement song learning. The current exercise design includes guided vocal patterns and timed breathing or lip-trill activities where available in the active build. Exercise scoring and pitch feedback should be documented according to the implemented behaviour. Any exercise that does not currently analyse microphone input must not be described as receiving automatic vocal evaluation.

5.9 Profile, Authentication and Persistence

Supabase is used for authentication and the configured persistence layer. The data model is intended to associate user profiles, vocal-profile information, songs, practice sessions and recordings with an account. The final report should describe only the tables, storage buckets and policies that are present in the submitted implementation. Access control must be verified at the backend and storage layers before the system is deployed for general use.



5.4 profile authentication

5.10 Error Handling and User Experience

The application must handle unsupported files, failed uploads, unavailable backend services, missing microphone permissions, empty pitch detections and interrupted playback. Clear loading states and error messages are important because audio-processing tasks may take longer than ordinary interface operations. The learner should be able to understand whether the application is processing, ready for practice or waiting for a new input.

6. METHODOLOGY AND ALGORITHMS

6.1 Overview of the Methodology

SVARA uses a sequence of audio-processing stages to transform an imported song and a live microphone recording into information that can support practice. The stages include audio input, source separation, reference-pitch extraction, lyric transcription, user-pitch estimation and comparison. The methodology uses established algorithms and pretrained models; the integration and user workflow are the main system-level contribution.

6.2 Audio Input and Pre-processing

The user selects an audio file supported by the application. The backend validates the input and prepares it for processing. Audio may be decoded or converted into a suitable format using FFmpeg or compatible libraries. A SHA-256 hash of the input content can be used as a cache key so that previously processed audio can be reused when the same content is submitted again.

6.3 Music Source Separation

Demucs is used to separate the mixed music signal into broad component stems. In a typical source-separation workflow, the model receives the mixed waveform and produces estimates of the vocal and accompaniment-related sources. The vocal output can then be processed further by the configured UVR/MDX-Net-family model. Separation quality depends on the input mix and the model; artefacts, leakage and incomplete isolation may remain.

6.4 Reference Pitch Extraction Using pYIN

The reference vocal stem is analysed to estimate its fundamental frequency over successive audio frames. The pYIN method evaluates candidate pitch periods and uses probabilistic temporal decoding to generate a pitch track. Frames that are unvoiced or have insufficient confidence can be excluded from the displayed

reference. The extracted pitch sequence is associated with timestamps so that it can be presented alongside song playback.

The pYIN output is an estimate of the reference pitch and may contain errors due to harmonics, vibrato, vocal effects, background sound and source-separation artefacts. The final application should preserve confidence information and avoid treating every frame as equally reliable.

6.5 Lyric Transcription and Timestamping

The isolated vocal audio is submitted to faster-whisper for transcription. Word-level timestamps are used to align recognised text with playback. Language hints may be supplied for the intended language. Since the input is singing rather than ordinary speech, the generated text and timing may be imperfect. Recognition accuracy and timestamp deviation should be measured using a curated set of sung examples before quantitative claims are made.

6.6 User Pitch Estimation Using YIN

During practice, the application captures microphone audio and estimates the fundamental frequency of the learner's voice. The custom YIN-based implementation evaluates periodicity in the input frame and returns a candidate pitch and confidence value. Frames with insufficient signal energy or low confidence should be treated as unvoiced or unreliable. The resulting user-pitch sequence is time-aligned with the practice session for comparison with the reference.

6.7 Pitch Comparison and Feedback

The reference and user pitch tracks are compared at corresponding times or within the available alignment window. When both tracks contain reliable voiced estimates, the difference can be expressed in semitones or cents. A positive pitch difference indicates that the user estimate is above the reference; a negative difference indicates that it is below. The feedback interface can translate this

difference into simple guidance such as “too high”, “too low” or “close to the reference”.

The exact feedback thresholds and scoring formula must match the version of the implementation used for evaluation. These thresholds are engineering parameters and should not be described as validated musical or pedagogical standards. Octave errors must be considered because a pitch estimator may identify a harmonic rather than the perceived fundamental.

6.8 Vocal-Range Estimation

The vocal-range workflow presents a sequence of target notes and evaluates whether the user can produce a stable pitch near each target. In the inspected implementation draft, a candidate note is accepted when the estimated pitch falls within 55 cents of the target for three consecutive frames, with up to two attempts per note. The scan uses a defined range of MIDI note values and records the lowest and highest accepted notes.

These values are implementation parameters, not validated clinical or pedagogical thresholds. A user's comfortable singing range can differ from the range of notes that can be produced briefly. Repeated testing and review by a vocal instructor would be needed to evaluate the reliability of the result.

6.9 Karaoke Playback and Transposition

After stem generation, the frontend loads the available audio outputs and presents playback controls. The user can adjust the stem levels and use supported speed or transpose controls. Transposition changes the pitch of the audio while attempting to preserve playback duration, depending on the processing method. The reference-pitch analysis must correspond to the audio actually played to the learner; otherwise, the displayed target may not match a transposed track. This alignment should be explicitly tested.

6.10 Data Caching and Persistence

A SHA-256 content hash can be used to identify previously processed audio and reduce repeated computation. Cached results must be associated with valid job metadata and protected from access by unauthorised users. Account-linked profile and practice information is stored through the configured Supabase services. The final implementation should document which information is stored persistently and which values exist only temporarily in application memory.

6.11 Algorithm Parameters

Component	Parameter / Method	Implementation Note
Vocal-range test	4096-sample buffers; nominal 44.1 kHz	Confirm against the final mobile build.
Range acceptance	Within 55 cents for 3 consecutive frames; up to 2 attempts	Implementation threshold; not independently validated.
Reference pitch	librosa pYIN; nominal 65–1000 Hz; hop length 1024	Retain voiced frames using configured confidence.
User pitch	Custom YIN implementation	Thresholds and frame settings should be versioned.
Lyrics	faster-whisper large-v3; word timestamps	Sung transcription may contain errors.
Audio separation	Demucs followed by configured UVR/MDX-Net-family processing	Exact checkpoint and version should be recorded.

6.12 Summary

The methodology connects audio preparation, pitch estimation, transcription and user feedback. The accuracy of each stage must be evaluated independently as well as within the complete workflow.

7. TESTING AND RESULTS

7.1 Testing Strategy

Testing should examine the functional behaviour of the application, the correctness of the audio-processing pipeline, the stability of microphone-based pitch estimation and the usability of the practice workflow. Functional testing verifies whether each module performs its intended operation. Technical evaluation measures pitch and transcription behaviour against suitable reference data. User evaluation examines whether participants can understand and use the application.

The project has been explored by approximately 20–30 users from varied backgrounds, including students, teachers, music teachers, amateur users and singers, with feedback also obtained from a sound engineer. This should be reported as preliminary user exposure or formative feedback, not as a controlled experiment. Insert the exact participant count, dates, protocol, consent process and recorded observations before submission.

7.2 Experimental Setup

The application is tested using a mobile device with microphone and audio playback capability, a development computer running the backend services and a set of audio files selected for karaoke and learning workflows. Testing should record the device model, operating-system version, application build, backend version, model checkpoints and relevant audio properties. For reproducibility, each test input should be identified without exposing personal recordings.

7.3 Functional Testing

Test ID	Test Case	Expected Result	Observed Result / Status
TC-01	Launch application and navigate modules	Main screens open without navigation failure	[ENTER RESULT]
TC-02	Import a supported audio file	File is accepted and processing can begin	[ENTER RESULT]
TC-03	Request source separation	Backend returns available stems or a clear error	[ENTER RESULT]
TC-04	Adjust stem volume and mute controls	Playback reflects selected mixer settings	[ENTER RESULT]
TC-05	Generate lyrics and timestamps	Recognised words and timestamps are displayed	[ENTER RESULT]
TC-06	Display reference pitch	Reference pitch is visualised against playback time	[ENTER RESULT]
TC-07	Record user singing	Microphone audio is captured and available for analysis	[ENTER RESULT]
TC-08	Display pitch comparison feedback	Reliable frames produce understandable feedback	[ENTER RESULT]
TC-09	Save or revisit practice data	Configured records are stored and retrievable	[ENTER RESULT]
TC-10	Handle microphone or network failure	User receives a clear error or permission prompt	[ENTER RESULT]

7.4 Vocal Analysis Evaluation

Vocal analysis should be evaluated by comparing the estimated lowest and highest notes with repeated measurements made under a defined protocol. Where possible, a trained vocal instructor should review the reference range. Report note-level agreement, repeatability across sessions, failed detections and the conditions under which errors occur. Do not treat a single successful note detection as proof of a stable vocal range.

7.5 Pitch Tracking Evaluation

A pitch-tracking evaluation should use a set of sung recordings with reference annotations or a suitable trusted reference method. Useful measures include voiced-frame coverage, median absolute pitch error in cents, gross pitch-error rate and octave-error rate. Results should be separated by register, accompaniment condition, vibrato, recording quality and singer experience where the dataset permits.

7.6 Source Separation and Karaoke Evaluation

Source separation should be assessed using listening tests and, where reference stems are available, objective metrics such as SDR or SI-SDR. Listening tests should document whether vocals remain in the accompaniment, whether instrumental components leak into the vocal stem and whether the resulting tracks are usable for practice. A user's preference for a particular stem mix can be recorded separately from objective separation quality.

7.7 Lyric Recognition and Alignment Evaluation

Sung lyric transcription should be tested on a multilingual sample containing the languages supported by the application. Word error rate may be calculated where a verified transcript is available, and timestamp deviation may be measured against manually reviewed word boundaries. Speech-recognition performance on ordinary speech should not be used as a substitute for sung-lyric evaluation. Do not claim perfect recognition for Tamil, Malayalam, Telugu, Hindi or English.

7.8 Preliminary User Feedback

The application has been explored by approximately 20–30 people across different user groups, including students, teachers, music teachers, amateur users and singers, as well as a sound engineer. This range of participants can provide useful formative feedback on navigation, clarity of pitch feedback, usefulness of karaoke controls and the overall practice workflow. The final report should include only observations that were actually recorded or can be substantiated from participant feedback.

Evaluation Area	What to Record	Actual Observation
Ease of navigation	Whether users could locate the main modules	[INSERT RECORDED FEEDBACK]
Karaoke controls	Clarity of stem, volume and playback controls	[INSERT RECORDED FEEDBACK]
Lyrics	Readability and perceived timing quality	[INSERT RECORDED FEEDBACK]
Pitch display	Whether users understood reference and user pitch lines	[INSERT RECORDED FEEDBACK]
Learning workflow	Whether the workflow supported repeated practice	[INSERT RECORDED FEEDBACK]
Overall usability	Common strengths, confusion points and suggestions	[INSERT RECORDED FEEDBACK]

7.9 Discussion

The main engineering contribution of SVARA is the integration of multiple stages of singing practice into one mobile workflow. A vocal profile can contextualise practice; song separation can make accompaniment more controllable; lyrics and pitch tracks can support song learning; and recordings can provide material for later review. However, integration alone does not demonstrate superior accuracy or

improved singing outcomes. Such conclusions require controlled and reproducible evaluation.

7.10 Limitations

- Pitch estimates may be affected by noise, harmonics, vibrato, breathiness and overlapping audio.
- Reference-pitch extraction depends on the quality of the separated vocal stem.
- Automatic transcription and word timing may be inaccurate for sung multilingual content.
- Any finer lead/harmony/ad-lib division based on thresholds is heuristic unless validated independently.
- Voice-type labels and pitch-feedback bands are approximate engineering rules.
- User feedback from a small formative group does not establish general effectiveness or learning improvement.
- Model versions, processing settings and backend security must be documented and tested for reproducibility and safe deployment.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

SVARA: Karaoke and Learn is an integrated mobile application designed to connect vocal analysis, karaoke preparation and guided singing practice. The system combines a Flutter frontend, a Python/FastAPI backend and Supabase-based authentication and persistence. Its audio workflow uses established technologies for source separation, reference-pitch estimation, lyric transcription and user-pitch analysis.

The project's principal contribution is the integration of these components into a common learner workflow. Users can explore an approximate vocal range, prepare a song for karaoke, follow lyrics and reference-pitch information, record their singing and access guided practice activities. The system is intended as a practice-support tool rather than a replacement for formal vocal instruction.

The quality of pitch tracking, lyric timing, audio separation and learning outcomes must be established through systematic evaluation. The application's current implementation and user feedback provide a foundation for further refinement, but quantitative claims should only be made when supported by documented measurements.

8.2 Future Scope

- Improve confidence-aware pitch feedback so that unreliable frames are not presented as definite pitch errors.
- Validate vocal-range estimates through repeated trials and review by trained vocal instructors.
- Improve reference-pitch alignment when users transpose or change playback speed.
- Evaluate source separation using listening protocols and objective metrics where reference stems are available.
- Improve multilingual sung-lyric recognition and word-level timing using verified datasets.

- Strengthen backend authentication, job ownership, file validation and secure recording access.
- Improve long-term persistence of practice history and progress information.
- Conduct a structured usability study and, separately, a controlled study of learning outcomes.
- Record exact model versions, checkpoints, package versions and processing parameters for reproducibility.

8.3 Final Remarks

SVARA demonstrates how mobile application development and audio-processing technologies can be combined to create a connected environment for singing practice. Its future development will focus on improving reliability, accessibility, evaluation quality and the usefulness of feedback for learners.

REFERENCES

[1] A. de Cheveigné and H. Kawahara, “YIN, a fundamental frequency estimator for speech and music,” The Journal of the Acoustical Society of America, vol. 111, no. 4, pp. 1917–1930, 2002, doi: 10.1121/1.1458024.

[2] M. Mauch and S. Dixon, “pYIN: A fundamental frequency estimator using probabilistic threshold distributions,” in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2014, pp. 659–663, doi: 10.1109/ICASSP.2014.6853678.

[3] A. Défossez, N. Usunier, L. Bottou, and F. Bach, “Music source separation in the waveform domain,” arXiv:1911.13254, 2019. [Online]. Available: https://arxiv.org/abs/1911.13254

[4] A. Rouard, F. Massa, and A. Défossez, “Hybrid Transformers for Music Source Separation,” arXiv:2211.08553, 2022. [Online]. Available: https://arxiv.org/abs/2211.08553

[5] A. Radford et al., “Robust speech recognition via large-scale weak supervision,” in Proc. International Conference on Machine Learning, PMLR, vol. 202, pp. 28492–28518, 2023. [Online]. Available: https://proceedings.mlr.press/v202/radford23a

[6] B. McFee et al., “librosa: Audio and music signal analysis in Python,” in Proc. 14th Python in Science Conference, 2015, pp. 18–24, doi: 10.25080/Majora-7b98e3ed-003.

[7] D. Hoppe, M. Sadakata, and P. Desain, “Development of real-time visual feedback assistance in singing training: A review,” Journal of Computer Assisted Learning, vol. 22, no. 4, pp. 308–316, 2006, doi: 10.1111/j.1365-2729.2006.00178.x.

[8] H.-Y. Chen, X. Chen, and J.-S. R. Jang, “Singer separation for karaoke content generation,” arXiv:2110.06707, 2021. [Online]. Available: https://arxiv.org/abs/2110.06707

[9] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[10] Flutter, “Flutter Documentation.” [Online]. Available: https://docs.flutter.dev/

[11] Supabase, “Supabase Documentation.” [Online]. Available: https://supabase.com/docs

[12] FFmpeg, “FFmpeg Documentation.” [Online]. Available: https://ffmpeg.org/documentation.html